iteration is the first internal node in the heap, at index 2 in our array. In the bottom part of the leftmost figure, it is pointed at by a red arrow. We repair the heap properties for the sub-heap rooted at this node by swapping its content with its child's.
(Step 3) We move our red arrow one position to the left, and repeat the operation for the sub-heap rooted at index 1.
(Step 4) Finally, the whole array is *heapified*, by pushing down the temporary root (with priority 5) until we find its right place.

Is this the best we can do? Turns out that we can do better.

Suppose that we initialize the heap with the whole sets of n elements, in any order.

As we have seen, each position of the array can be seen as the root of a sub-heap. So, leaves, for examples, are trivial sub-heaps, containing only one element. Being singletons, therefore, they are valid heaps.

How many leaves there are in a heap? That depends on the branching factor. In a binary heap, half the nodes are leaves. In a 4-way heap, the last three fourths of the array contain leaves.

Let's restrict to the case of a binary heap, for the sake of simplicity: we can see an example for a min-heap in figure 2.12, where we can follow step by step how `heapify` works.

If we start at the last internal node for the heap, let's call it `x`, it will have at most 2, children, both leaves, and hence both